Stack Overflow Big Data Analysis

Ibrahim Ali
Rajan Grewal
Alex Sichitiu
Arshdeep Chhokar

Problem Definition

The founders and stakeholders of Stack Overflow have grown their user base by a considerable amount over the course of the last 8 years (2008 - 2016). This also comes with an exponential growth of their databases containing questions, answers, and tags data. As a result, they want to move from their traditional database and data warehouse design to a more flexible data lake architecture. The primary motivation behind this is to support unstructured and semi-structured data and also scale up to larger datasets in the future. Currently, their data is located in three CSV files which contain data on questions, answers, and tags respectively.

As data engineers, we were hired by Stack Overflow with the main task of building an ETL pipeline that is responsible for extracting their data, transforming it and performing analyses on it, and lastly loading it into a destination store in a prettier data format. From here, we are able to directly query this data to support business analysts in their continuous search for patterns and insights into users on the platform. These insights are then gathered to create the annual Stack Overflow developer survey to better understand the tech community.

Essentially, we can run our ETL pipeline and then execute queries that are aimed to answer important business questions related to user engagement, distinguished users, technology trends, and response speed. Once we have our results, we can then visualize this data in a dashboard to see if it truly matches the expectations of the analytics team at Stack Overflow.

In addition, Stack Overflow would like to develop an automatic tag generation tool that can scan the text of a question and generate an associated tag. They want an initial proof of concept which makes these predictions to an adequate accuracy first. Our task is to implement, train, and test a machine learning model that can make these predictions using a natural language processing tool.

Of course, this undertaking does not come without many challenges. Namely, the challenge of processing a large volume of historical data with cost and scalability in mind, constructing a performant data pipeline that runs in the cloud, and experimenting with a machine learning model that produces tag predictions robustly and accurately despite the hurdles of processing natural language.

Methodology

Data Pipeline

The starting point for our data pipeline was the three S3 buckets which contained the CSV files for questions, answers, and tags data. Our next step was to start up an EMR cluster and run five steps: one for each Spark script performing an analysis plus the ETL script. Each script writes the final data frames to parquet files which are then stored in destination S3 buckets. At this point, our data is ready to be queried by Athena. During this phase, we had to create tables that matched the schemas of our data frames from each analysis and copy the data from our S3 buckets into these tables. After this was done, our data was ready to be accessed from QuickSight for visualization. We designed the pipeline this way to satisfy Stack Overflows requirements for a scalable and flexible data lake design. Using S3 for storage we can handle large amounts of structured and semi-structured data. In addition, by storing our results in Parquet format, we are able to leverage a columnar orientation for better querying of the data. Lastly, the use of Athena allows us to query the data directly from S3 without having to set up a complicated database server such as Postgres or MySQL.

Data Cleaning

This ETL process emphasizes data reduction and optimization by leveraging targeted filtering, schema enforcement, and efficient transformations during data processing. By focusing only on the top 10 most frequent tags, irrelevant data is excluded early, reducing the dataset size and improving processing efficiency. Unnecessary columns such as Title, Body, and ClosedDate in questions and Body in answers are dropped to streamline the data structure and minimize storage and computational overhead. Invalid answer records—those posted before their corresponding questions—are eliminated to ensure data integrity. Additionally, repartitioning the data by tags enhances the efficiency of subsequent group-by operations. The final datasets are stored in Parquet format, which further optimizes storage and query performance through compression and columnar storage. These techniques collectively enhance both the processing speed and scalability of the ETL pipeline.

Answer Speed

The goal of this analysis is to find the average time it takes for an answer to be posted on the top ten tags. There is a question to consider here, that is, what is exactly considered as an "answer"? Generally, it would be considered as the response which receives the check mark from the original question poster. However, upon examination of the data, it turns out that whether an answer was checkmarked or not is not included in the dataset. The question we are really answering in this analysis is: what is the average number of hours needed for an answer to be posted to a question, for each tag? Given this, we decided that it would be most appropriate to mark a response as an answer if its score is at least 3. Simply taking the first response wouldn't accurately capture the definition of an answer. Choosing a score that is too high would exclude too many responses and significantly reduce our sample size. There might be a better answer posted later, but we are interested in the first answer that is considered "good enough." We take the first response that meets this criteria, and the difference between its date and the date the question was posted. Then we simply average these numbers across the entire tag, and do this for each of the top ten tags.

Tags Over Time

The goal of the "top tags by year" analysis is to identify trends in the popularity of the Top 10 most commonly used tags over time. The question being answered is: Which tags are most frequently used each year, and how does their usage change over time? This is achieved by first calculating the average number of questions for each tag per year. The top 10 tags, based on overall usage, are then identified and filtered to analyze their yearly trends. This allows us to observe whether certain tags remain consistently popular or if their usage fluctuates significantly over time.

The "analyze_tag_monthly_usage" analysis focuses on understanding the seasonal and monthly patterns in the usage of the top 10 tags. The key question here is: How does tag usage vary month-to-month, and are there any observable patterns? For each tag, the average number of questions is aggregated by year and month, providing a detailed view of how engagement changes over time. These insights can help identify when a tag is most active and whether its usage is subject to regular seasonal cycles.

User Contribution

Sites that host user-made content often have a skewed distribution of user contribution. For instance, there may be power users (e.g. the top 10% most active users create 50% of the overall content). To better understand the distribution, we related how much of the content the top n% of users contribute for a range of different n. This is only defined for the users that actually ask questions or give answers as we only know about users present in the question and answer data. This will be done independently for both questions asked and answers given as the content of interest.

To achieve this, we will use question and answer data to get the total number of questions a given user has asked or answers they have given, sort the users by this in descending order and calculate a running sum of the total questions asked or answers given by the top n users. This can be divided by the total number of questions asked or answers given to determine what fraction of the overall content that the top nth percentile of users has contributed.

User Engagement

In order to identify the engagement of users on the platform, we focused solely on the top 10 tags associated with both questions and answers. We created two data frames grouped by tags: one that calculates the average scores of questions and another that calculates the average scores of answers. These two data frames will help us measure the quality and usefulness of questions to stack overflow users. In addition, we also created data frames for the total number of questions and the total number of answers from the top 10 tags to see how active the community is on certain topics. Once these four data frames were created, we then combined them into a single data frame which represents how engaging and popular each tag is. Using this data frame, we can figure out which tools and technologies are getting adequate attention, and which ones require further attention and stronger support from the community.

Tag Prediction

We wanted to create a tool that could predict tags based on the question body. One application of this would be to retroactively apply tags to old question data for analysis. Another would be to use this model as a feature that automatically tags new questions for user convenience.

Using the Spark NLP 5.5.1 library, we encoded the question body with the pretrained UniversalSentenceEncoder and fit a MultiClassifierDL model onto our question data. Since questions can have any number of tags, we chose this model as it could assign multiple classes to a single input.

We chose SparkNLP to have a model that can be trained and stored distributively. This allows training to scale with our data as questions are expected to scale rapidly and body text data can be quite large.

Problems

Answer Speed

This is briefly mentioned in the methodology section as well, but will be covered again for completeness. There was a problem in that there was no way to get a real answer, that is, a response that received a check mark from the original question poster. We worked around this by just taking the first answer to have a score of at least three. This is generally good enough, and in some cases, could be even better in the sense that sometimes the question poster will mark something as an answer, even though it isn't a particularly good answer.

User Contribution

The initial approach used a Spark Window over all the users sorted by the total number of questions or answers they gave. This was done to get a cumulative sum of the questions asked by the top n users for each row which would be used to determine the fraction contributed by the top n users. However, this coalesces the DataFrame (which has as many rows as there are users) to a single partition, undermining the scalability of the analysis.

We instead grouped users by the number of questions asked or answers given. We could know how many questions were asked by users who had each asked a count of n questions (e.g. a total of 2000 questions was asked by 80 users who asked 25 questions each. The count equals 25). While this also used a single partition Window to calculate the cumulative sums, the DataFrame was only as long as the range from 1 to the maximum count value. This scales much slower than the total number of users. For example, there could easily be a million users, but it is unlikely that there will be a user that asks a million questions. Even if such a user existed, this would create only one row for a count of one million. To have a row for a count of 999,999 questions, there would need to be another user that had asked that many.

We then used a builtin DataFrame method to get the percentile scores for the count of questions asked or answers given by an individual user. We could then join this against the above. This reduced the granularity of the results as the same count would appear at different percentiles. Let's assume that I calculated all the integer percentiles. The 1st through 59th percentiles would all have a count of one question asked. This is problematic for calculating a cumulative sum because say, the 40th percentile excludes some of the users who asked only one question. However, I only know the total number of questions asked by *all* users with a count of one. This corresponds to the most inclusive percentile, which is the lowest. Consequently, I filtered all but the lowest percentile associated for a given question count (e.g. everything between 59 to 2 was ignored in favor of the 1st percentile for a question/answer count of 1), reducing the number of percentiles available.

This also creates some ambiguity. For example, the 60th percentile may be the smallest percentile that has a breakpoint of count two. However, the true breakpoint may be between 59 and 60 (e.g. 59.7). Consequently, I calculated the percentiles at intervals of 0.01% to improve the estimation accuracy.

Tag Prediction

While running SparkNLP code locally in WSL2 on the entire dataset, the process would often crash with an error stating "answer from Java side was empty." The likely cause was the JVM running out of memory. In response, we had to increase the RAM and swap space available to WSL2 to 16 GB and 8 GB, respectively. This is less likely to be an issue when run on a true cluster.

Working with AWS

We faced some challenges when working with EMR. Namely, anytime a bug was found in the code, our teammates would need to fix it and this required a re-run of the associated step in EMR as well as re-creation of the table in Athena. Similarly, if the ETL script itself required a bug fix, we would need to re-run every step of our analysis as well as re-creating all Athena tables from scratch. This was a somewhat painful and tedious part of our experience working in the cloud.

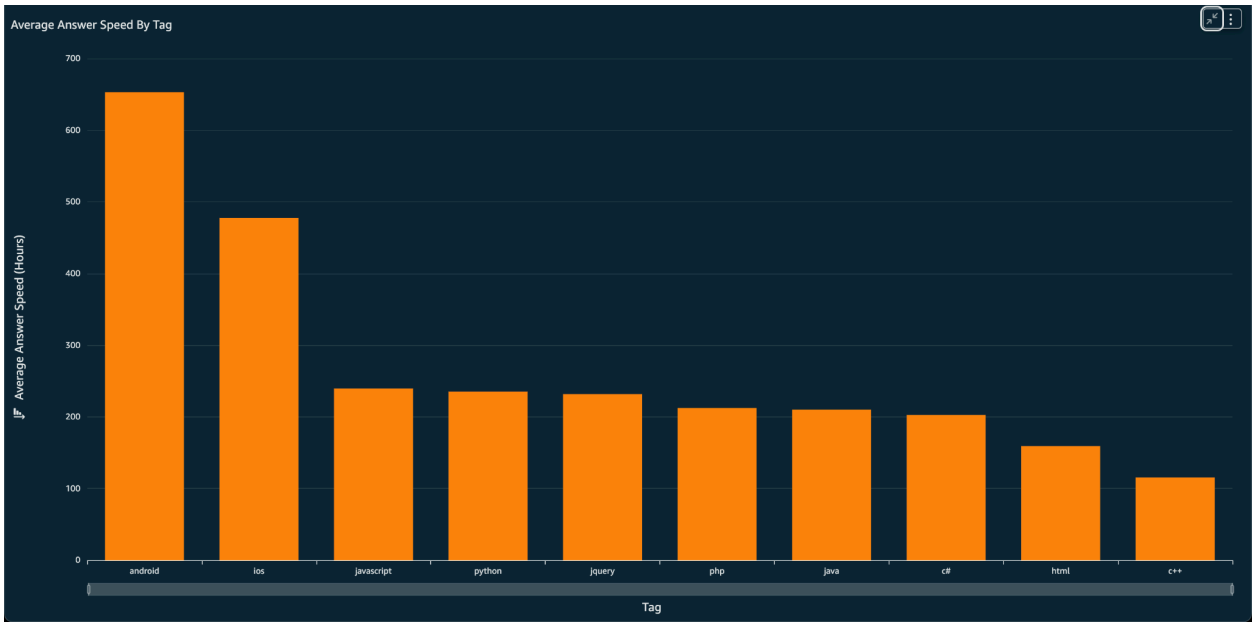
In addition, when setting up our QuickSight dashboard, we were not able to find a way to make it publicly accessible without having to pay Amazon a handsome fee of $250 dollars for the month. In order to work around this, the solution was to create temporary IAM users for the TAs with all necessary permissions to gain full access to QuickSight. Since QuickSight depends on Athena which depends on S3, we had to create permissions for each of those services as well.

Results

AWS Data Pipeline

Leveraging AWS to build our ETL pipeline gave us some much needed experience with key services like EMR, S3, and Athena. One of the biggest lessons we learned from this implementation is how to structure our data in S3 in a format that is easy to query using tools such as Athena. Furthermore, by playing around with visualizations in QuickSight, we got a sense of the types of illustrations which would be most intuitive to stakeholders. Lastly, building this pipeline on AWS made us more conscious of the tradeoff between cost-efficiency and scalability.

Answer Speed



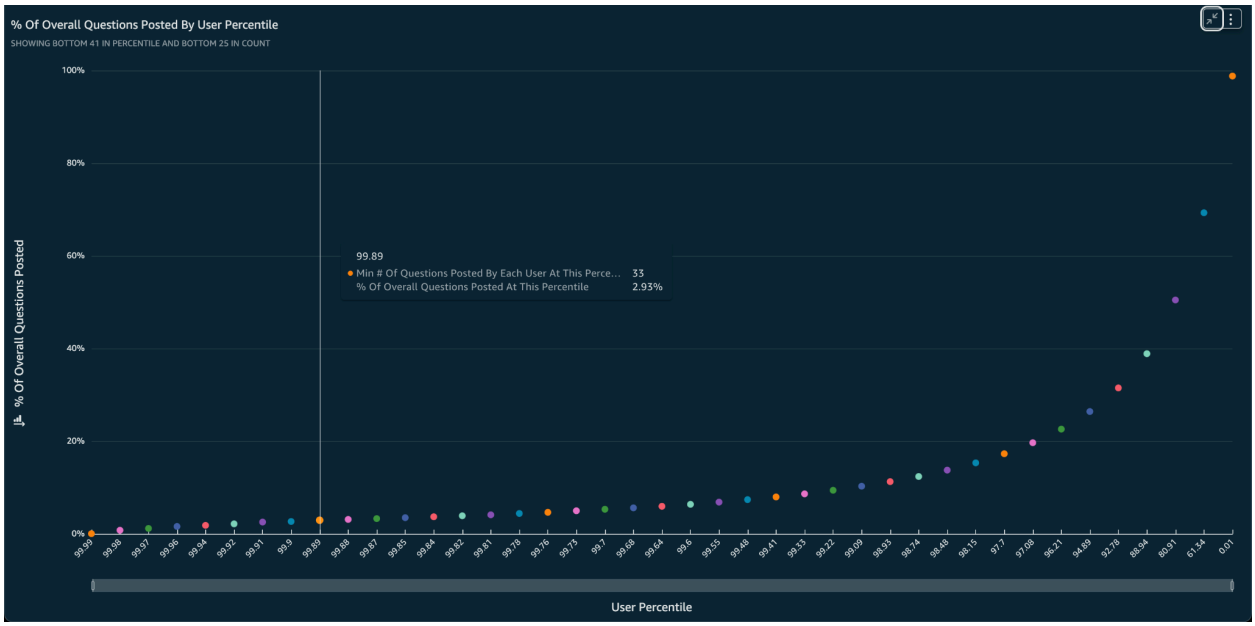
Looking at the results, we can see that the average response time is on the scale of weeks. For android it was as high as 653 hours for a response time. C++, on the other hand, had a much faster 115 hours. The results observed here were initially surprising. We did not think it would take so long for questions to be answered. Some data examination was conducted to see why there was such a high response time. We found out that there are quite a few cases where answers would hit a score of three significantly later than the date the question was posted. There are also cases of very late responses being posted, which end up being the first response to hit a score of three as well. We could have filtered out these outliers, and just kept answers that were posted no more than a week after, but this is really open to interpretation. Both could be seen as acceptable measures. Our logic is that we wanted to capture as much data as possible, so we opted for the given approach.

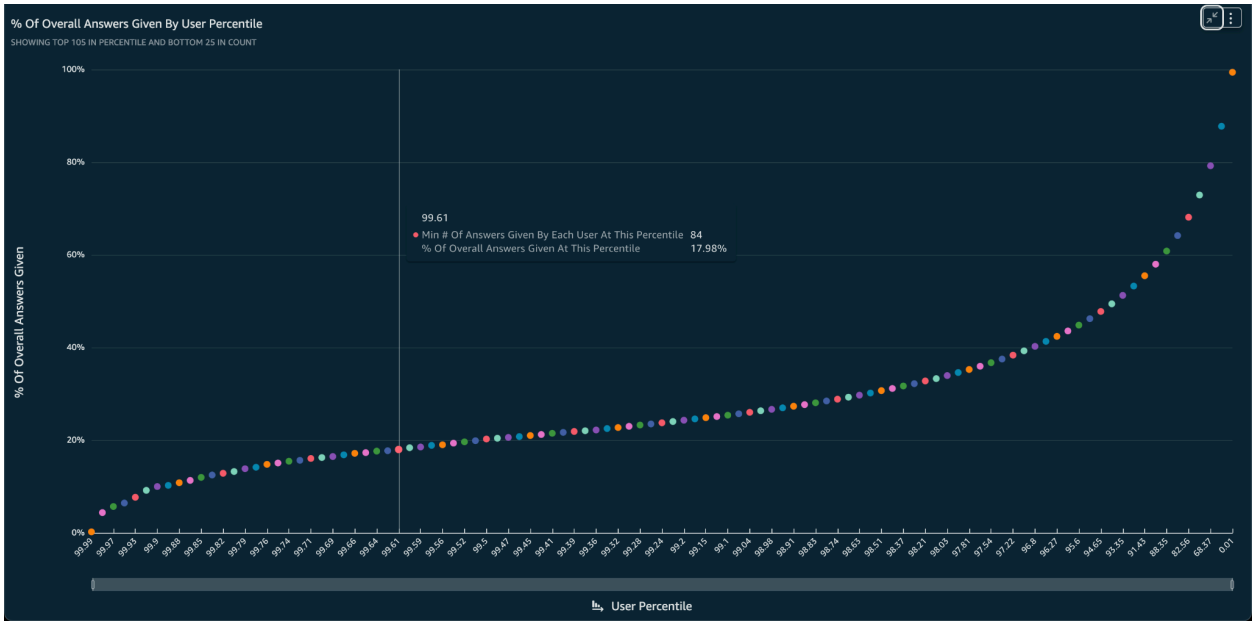
From conducting this analysis, I learned that many expectations may not match reality. Many times, one only knows the true answer to something in a big-data context after conducting a proper and thorough in-depth

analysis on the available data. Alongside that, I gained a lot of knowledge on how to carefully analyze data such that it matches the question which is trying to be answered. There can be several ways to conduct operations like this, but us as data scientists must determine which is the optimal route to take.

The implementation of the project gave me a solid overview on how data analyses are conducted from data gathering, all the way down to the visualization. There is a lot in between, and it is all there for a reason, that is, to produce the best, most reliable data.

User Contribution





As expected, the most active contributors of both questions and answers produced a disproportionately large amount of the total questions. Using linear interpolation, we can see the following:

Table 1: Overall Question and Answer Contributions by Percentile

Percentile	% of Overall Questions	% of Overall Answers
99	10.83	26.36
95	26.07	46.72
75	56.16	74.58

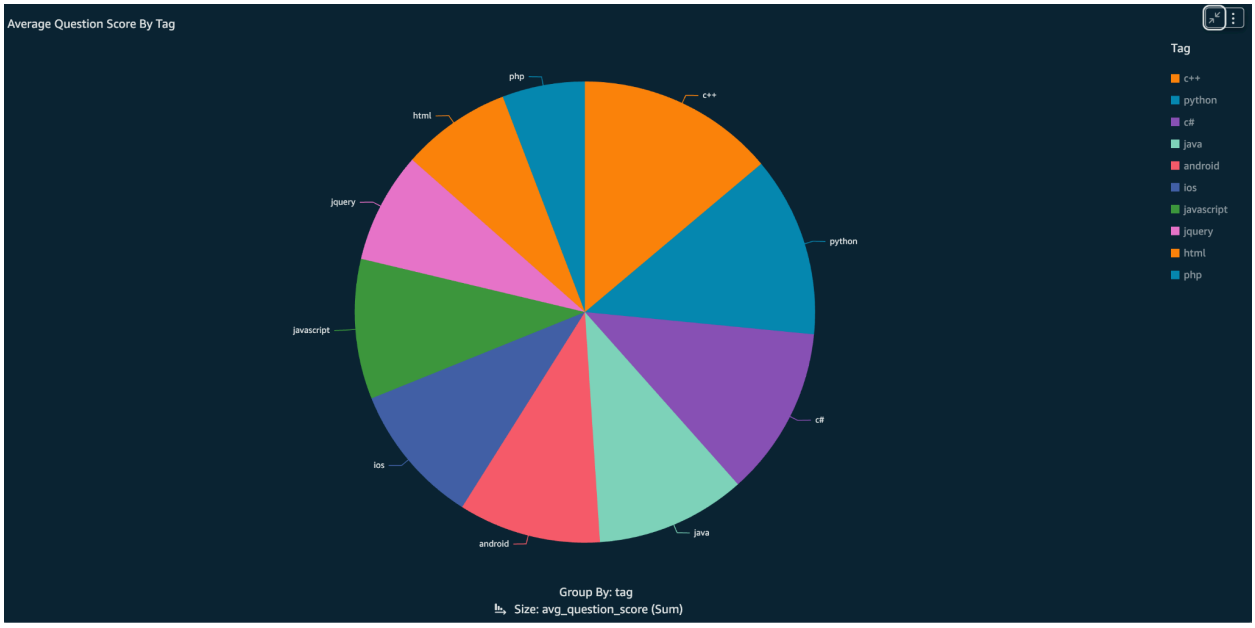
While there exists power users for both questions and answers, it appears that this trend is more prominent in answers given.

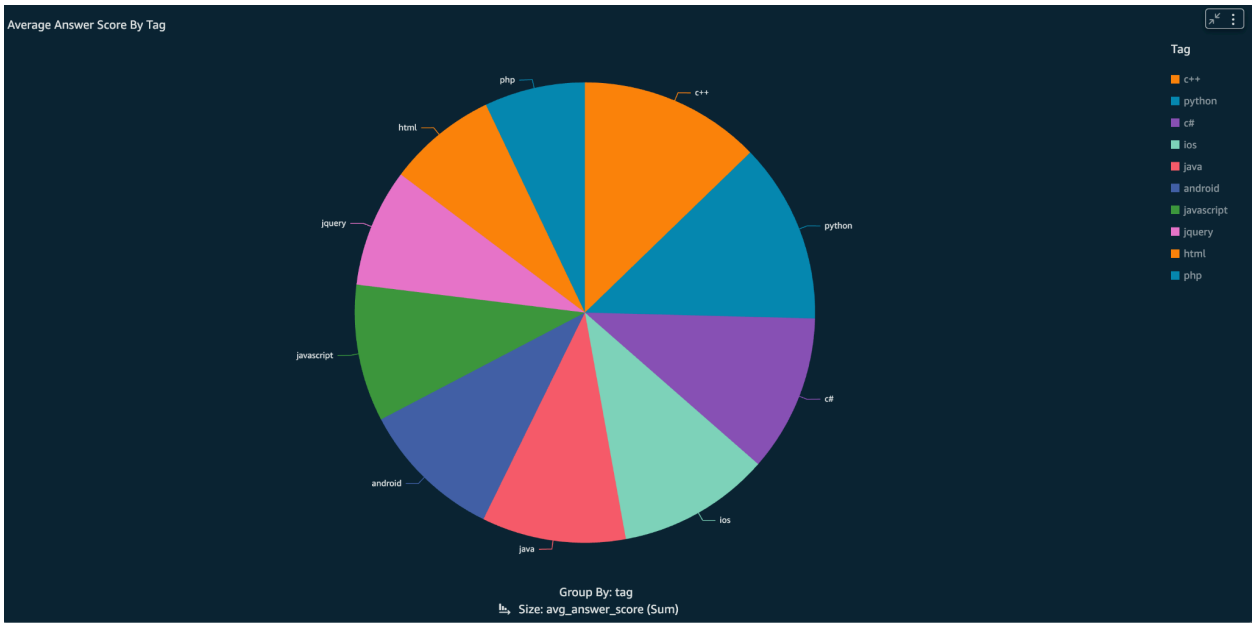
One note about the above graphs is that the lowest percentile does not reach 100% of answers or questions contributed because the dataset included questions and answers associated with no user. These instances are not accounted for in the user percentiles, but they are included in the total number of questions and answers. However, these constitute 1.2% of the question data and 0.6% of the answer data, making their impact negligible.

Another note is that these percentiles are with respect to users that have actually asked questions or given answers. If the users who didn't ask questions or give answers were included, they would be part of the bottom percentiles. This would map the current percentiles to higher percentages of content contributed and increase the importance of the most active users.

Given the data showing how important the most active users are for answers and questions, we recommend that Stack Overflow make efforts to court these users as they provide a significant amount of the site content.

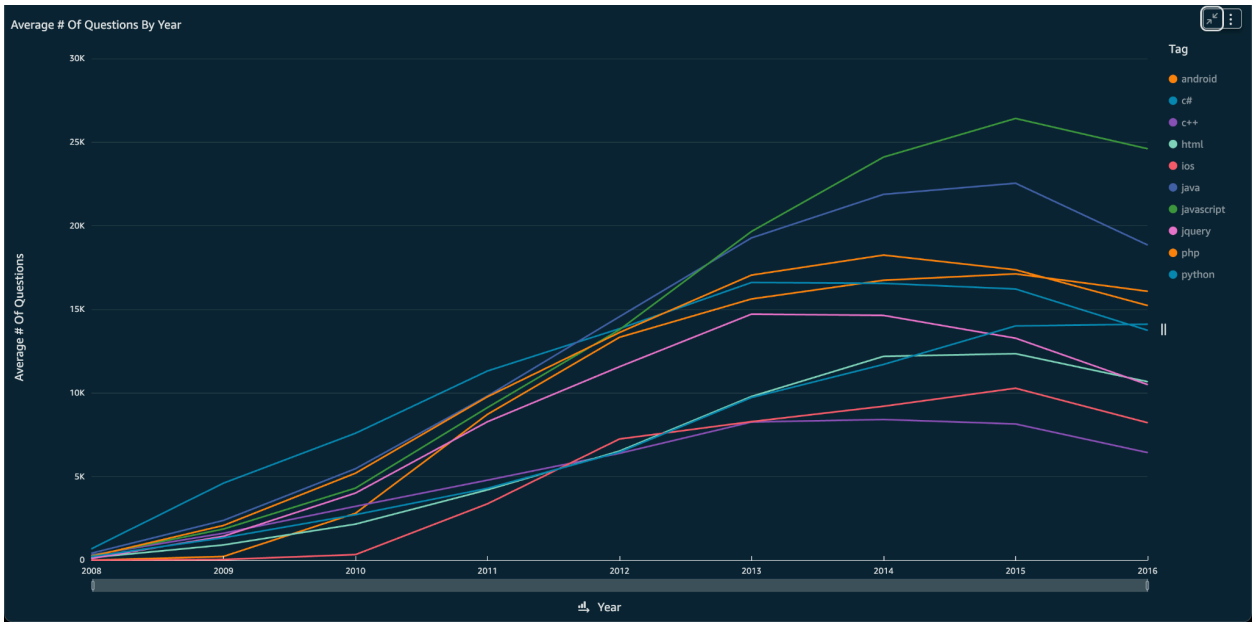
User Engagement





From the average question score by tag analysis, we can see which tags have the highest question scores. This reflects the level of curiosity and interest the community has for specific tools and technologies, as well as their ability to effectively articulate questions related to them. On the other hand, the average answer score by tag analysis reveals which tags have the highest answer scores. This gives us insights into the level of experience and support the community is able to provide for those particular tools and technologies. Looking at both analyses together, we can see a consistent pattern for tags like Python and C# which seem to maintain high question and answer scores. This suggests that these two technologies generate the most interest and curiosity in developers and also have the best community support. On the flipside, technologies such as PHP and HTML have lower scores which indicate a lack of quality contributions and a greater need for engagement with these languages.

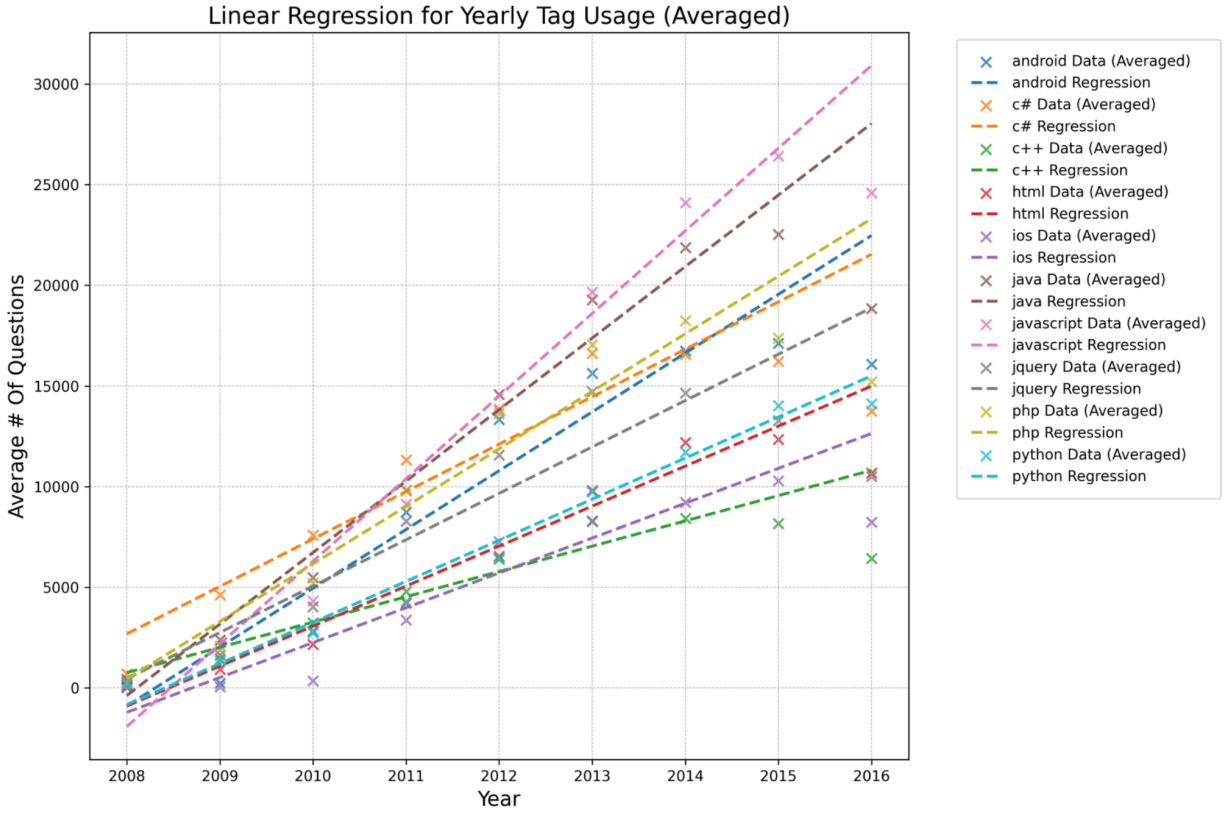
Avg # of Questions By Year



The analysis shows that the trend for all tags was upwards until 2015, reflecting increasing community engagement and interest in these technologies over time. However, in 2016, there is a noticeable dip across all tags. Several factors can explain the dip in 2016, including the following:

1. Newer Frameworks and tools: Technologies included in the top 10 tags, had matured and begun to be replaced by newer frameworks and tools. Users may have shifted to asking questions about tags that are not included in the top 10.
2. Newer Platforms or Forums: StackOverflow might have faced competition from newer platforms or forums, emerging in 2016 (eg. GitHub Discussion, Reddit and Quora)
3. Industry Trends in 2016: Mobile development started shifting from native frameworks to cross-platform tools like React Native. Similarly backend technologies faced competition with alternatives like Node.js and Python's Django.

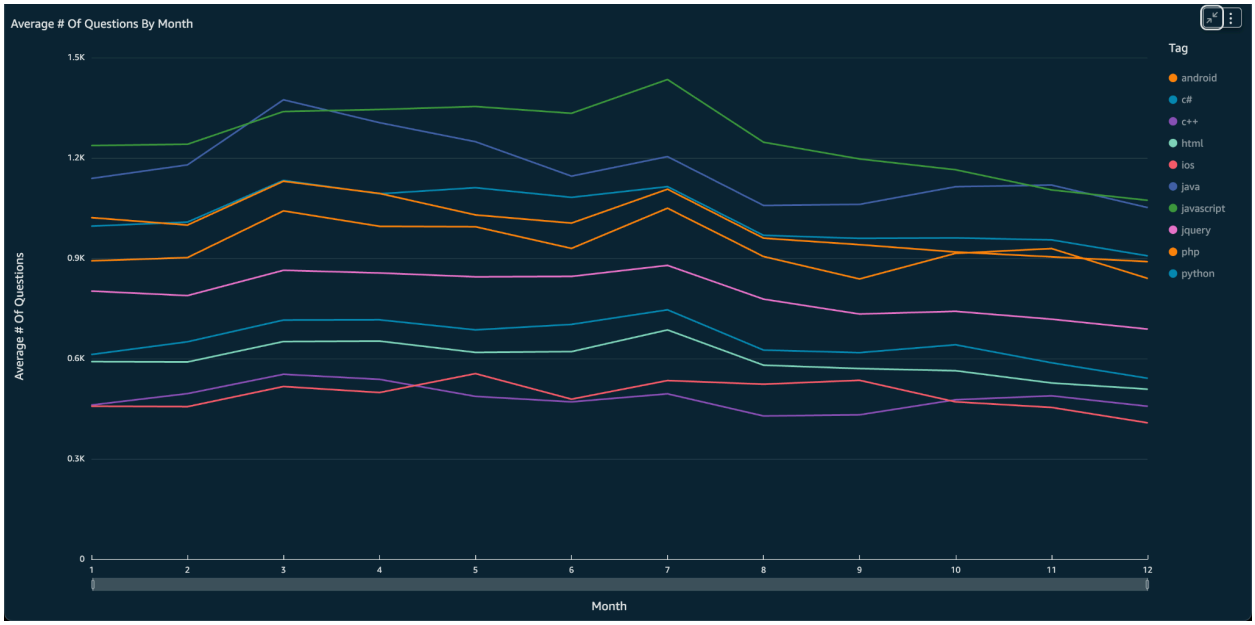
A linear regression model was applied to the data, the analysis provides insights into the average annual increase in the number of questions for each tag. The following plot shows the results:



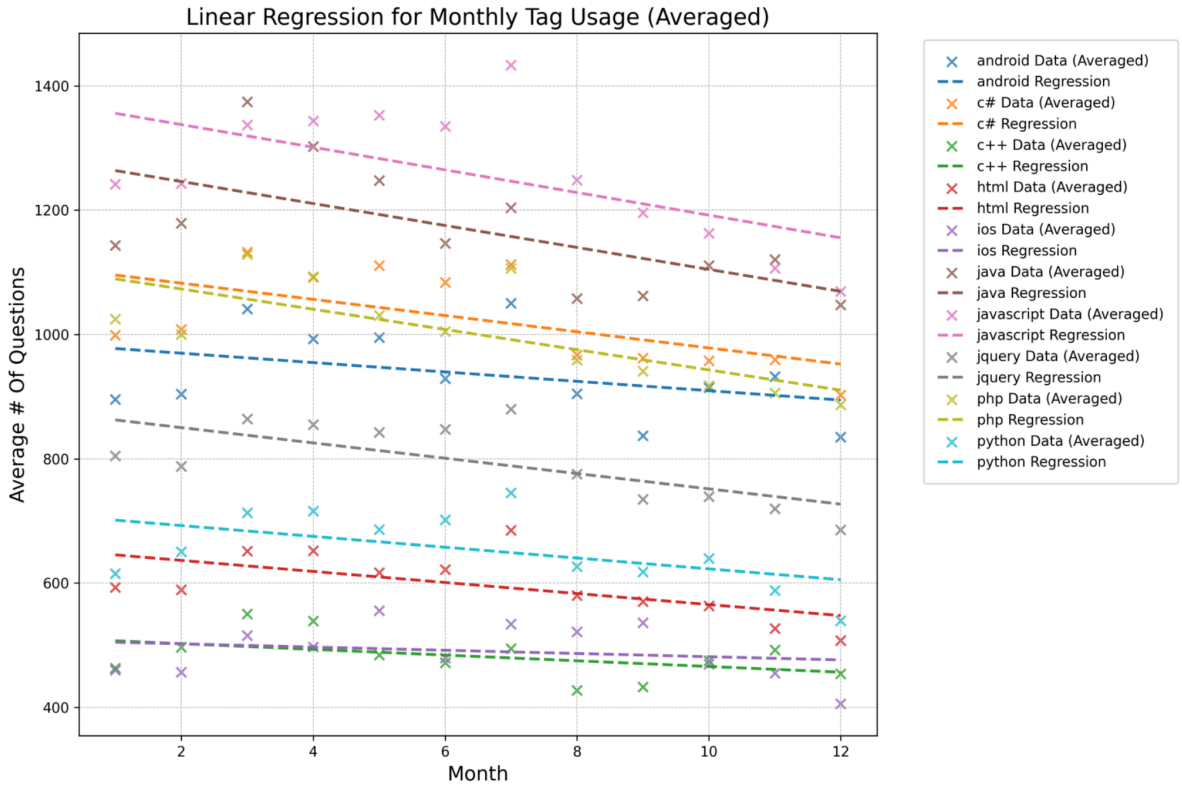
The slope represents the average annual increase in the number of questions for each tag. JavaScript showed the steepest growth, reflecting its popular demand in web development. C++ had the lowest growth rate, confirming that it is a more stable,mature technology with slower adoption.

In conclusion, while the growth in questions for all tags was steady and significant until 2015, the dip in 2016 underscores the impact of shifting industry trends, new frameworks, and the emergence of alternative platforms for developer engagement. The regression analysis further quantifies these trends, providing a detailed understanding of how different technologies evolved over the period.

Avg # of Questions By Month



The analysis of the average number of questions by month shows that question activity tends to decrease slightly across all tags as the month progresses. The slope represents the average monthly change in the number of questions, and the intercept reflects the estimated number of questions at the start of the year (Month = 1). The following plot shows the results:



The following information can be derived from the analysis:

1. All slopes are negative, indicating a gradual decline in the number of questions as the year progresses. This could be due to seasonal trends, with fewer questions during holiday months or when development activity slows down.

2. Highest Activity: JavaScript has the highest intercept (1374.03) and slope (-18.21), reflecting its dominance in the community but also showing the steepest monthly decline.
3. Least Decline: iOS has the smallest slope (-2.59), indicating relatively stable activity throughout the year compared to other tags.
4. Programming Stability: Tags like Python (-8.71) and HTML (-8.88) show consistent levels of engagement, with slower declines compared to more dynamic tags like Java (-17.67) and PHP (-16.28).

In conclusion, the analysis shows the importance of tracking monthly trends for understanding platform activity, user behavior and external factors influencing technology engagement.

Tag Prediction

The model we trained and saved had the following metrics on the test data, averaged over 5 epochs:

- Macro-average: Precision: 0.7815, Recall: 0.5042, F1: 0.6128
- Micro-average: Precision: 0.7688, Recall: 0.5111, F1: 0.6139

The macro-average is equally weighted across all classes whereas the micro-average is weighted according to the class frequency in the dataset. Overall, the model isn't very accurate, with a particularly low recall in both cases. This could likely be improved with more tuning of the model hyperparameters, but this is frankly beyond the scope of our knowledge. Nevertheless, this model stands as a proof-of-concept.

Project Summary

At the end of your project report, please provide a summary of the emphasis/priorities in your project. Give yourself a total of 20 point in these categories:

- Getting the data: Acquiring/gathering/downloading.
  - **Score - 1:** We acquired and downloaded our dataset from Kaggle
- ETL: Extract-Transform-Load work and cleaning the data set.
  - **Score - 4:** We did extensive work to clean, transform, and repartition our data appropriately
- Problem: Work on defining the problem itself and motivation for the analysis.
  - **Score - 3:** We framed our problem to be as realistic as possible and aligned our goals with real stakeholders in mind
- Algorithmic work: Work on the algorithms needed to work with the data, including integrating data mining and machine learning techniques.
  - **Score - 4:** We experimented with the "sparknlp" library to produce a working proof of concept for a tag prediction feature
- Bigness/parallelization: Efficiency of the analysis on a cluster, and scalability to larger data sets.
  - **Score - 2:** We took big data and turned it into relatively small data that can be stored and processed efficiently in the cloud
- UI: User interface to the results, possibly including web or data exploration frontends.
  - **Score - 0:** Although we have a dashboard, the UI interactions are very limited
- Visualization: Visualization of analysis results.
  - **Score - 3:** We tried to be thoughtful about which visualizations make the most sense for each analysis that we performed
- Technologies: New technologies learned as part of doing the project.
  - **Score - 3:** We learned a lot about running Spark Jobs in EMR, creating tables in Athena, building visualizations in QuickSight, and training models using sparknlp